

BMB 306

YAZILIM MÜHENDİSLİĞİ



Temel Kavramlar

Yazılım Mühendisliği

- ✎ Yazılım Mühendisliği, yazılım üretiminin mühendislik yöntemleriyle yapılmasını öngören ve bu yönde yöntem, araç, teknik ve metodolojiler üreten bir disiplindir.
- ✎ Bu bakımdan Yazılım Mühendisliği bir yöntemler kümesi, teknikler kümesi ya da araçlar kümesi olarak değerlendirilebilir.
- ✎ Yazılım üretiminde Yazılım Yaşam Döngüsünde belirtilen aşamaların sistematik olarak izlenmesi ve gerçekleştirilmesi yazılım mühendisliği için ön koşuldur.
- ✎ Yazılım mühendisliği, yazılım üretimindeki karmaşıklıkları gidermeyi hedefler.
- ✎ Geçmişte kullanılan iş akış şemaları gibi yöntemler, günümüzde, yazılım projelerinin boyutlarının büyümesinden dolayı yetersiz kalmıştır. Öte yandan artık yazılım üretim işi, tek kişinin başarabileceği boyuttan çıkmış ve takım işi biçimine dönüşmüştür. Yazılım mühendisliği bu olgular üzerine önem kazanmıştır.

Temel Kavramlar

- ☞ Günümüzde, doğrudan yazılım mühendisi mezun eden üniversite bölümleri yeni yeni oluşmaya başlamıştır. Daha çok yüksek lisans eğitimi ile başlayan Yazılım Mühendisliği disiplini, lisans düzeyine taşınmaktadır.
- ☞ "Yazılım Mühendisliği" ile "Mühendislik Yazılımı" terimlerinin karıştırılmaması gerekir. **Mühendislik yazılımları**, daha çok donanım bileşeni ile ilgisi olan (örneğin sürücüler vb.) yazılımlardır. **Yazılım mühendisliği** ise, yazılım mühendisi yetiştirmeyi amaçlayan bir disiplindir.

Temel Kavramlar

Yazılım Mühendisi

- ✎ Yazılım mühendisliği işini yapan kişidir.
- ✎ Değişik bilgisayar bilimi teknolojilerinin ve kişilerin bir bilgi ya da yazılım sistemi oluşturmak amacıyla biraraya getirilmesinde bir bütünleştirici gibi çalışır. Temel hedefi, söz konusu üretimin en az maliyet ve en yüksek kaliteli biçimde yapılmasıdır.
- ✎ Yazılım mühendisi bir programcı değildir. Ancak programcının tüm yeteneklerine sahiptir. Programcı, ağırlıklı olarak kodlama, sınama işi ile ilgilenir. Yani programlarla ilgilenir. Yazılım mühendisinin işi daha çok insanlarla ilişkiyi gerektirir. Yazılımın daha çok mantıksal boyutuyla ilgilenir.
- ✎ Yazılım mühendisi ile sistem çözümleyici arasındaki belirgin fark ise, sistem çözümleyicinin üretimin yalnızca çözümleme (analiz) aşamasında, yazılım mühendisinin ise tüm aşamalarında yer almasıdır.

PROJE YÖNETİMİ VE AŞAMALARI

- ∞ Analiz ve Planlama aşaması
- ∞ Gereksinimlerin belirlenmesi aşaması
- ∞ Ön ve detaylı tasarım aşaması
- ∞ Gerçekleme aşaması
- ∞ Test ve entegrasyon (birleştirme) aşaması
- ∞ Sonuçları değerlendirme aşaması
- ∞ Bakım aşaması

Analiz ve Planlama Aşaması

- ☞ Projeye başlarken öncelikle iyi bir analiz süreci yaşamak ve planlama yapmak gereklidir.
- ☞ Öncelikle müşterinin ne istediğinin, nasıl bir sistem istediğinin tam olarak anlaşılması gerekir.
- ☞ Müşterinin istediğinin ne olduğu tam olarak anlaşıldığında risk analizi ve maliyet tahminleri yapılır.
- ☞ Detaylı gereksinimler de analiz ve planlama aşamasında oluşur.
- ☞ Gereksinimler bu aşamadan sonra değişmeyecek diye bir kural yoktur, ancak bu aşamada ne kadar tutarlı bir gereksinim dokümanı oluşturulur ve ona sadık kalınmaya çalışılırsa proje de o kadar sağlıklı ve rahat gerçekleştirilir.
- ☞ Analiz ve planlama aşamasında sistemin nasıl test edileceğinin planlaması da yapılmalıdır. Test edilmesi zor veya test edilme imkanı bulunmayan bir sistem hiçbir zaman üretilmemelidir.
- ☞ Analiz ve planlama aşamasında bu konuların yanı sıra projeyi gerçekleştirecek takım da oluşturulur.

Analiz ve Planlama Aşaması

∞ Her Şey İş Tanımı İle Başlar

- İş tanımı, projedeki tüm taraflara ve projede çalışan elemanlara işin amacını ve kapsamını kabaca anlatan bir dokümandır.
- İş tanımı dokümanında şu iki konuya mutlaka değinilmelidir:
 - **İşin Amacı**
 - **İşin Kapsamı**
 - **Diğer Alanlar:**
 - İşin süresi
 - İşin yapılacağı yer
 - Teslim takvimi
 - Teslim kalemleri
 - Kabul kriterleri
 - Ödeme koşulları

Analiz ve Planlama Aşaması

∞ Projenin başarısındaki etkenlerden biri projenin amacının iyi anlaşılmasıdır

"Bir projeyi planlamaya başlamadan önce projeyi neden yaptığınızı, yapma sebeplerinizi iyice anlamalısınız. Projeye neden ihtiyacınız olduğunu kavramadan proje sonunda başarılı olup olmadığınızı nasıl söyleyebilirsiniz ki?"

Kent Beck, Martin Fowler

Analiz ve Planlama Aşaması

☞ Projede Maliyet ve Süre Tahminleri

- Bir projenin yapılma sebebi kar etmektir ve bunu mümkün olan en kısa sürede yapmaktır. Kar edilmeyecek bir projeye girmenin anlamı yoktur, bu sebeple maliyet ve süre tahminlerinin doğru yapılması çok önemlidir.
- **Proje süresi hesaplanırken genel eğilim iyimserliktir ve bu durum birçok projenin batmasına sebep olur.** – İyimserliğin genellikle iki kötü sonucu vardır:
 - Proje ekibi ne pahasına olursa olsun projeyi bitirir ama artık ekipten hayır gelmez.
 - Proje ekibi projeyi zamanında bitiremez, gecikme tazminatları, müşteri kaybı, ekipte fazla ve sürekli stres gibi birçok zarar oluşur.

"İyimserlik programlama işinin bir meslek riskidir ve tedavisi geribeslemedir."

Kent Beck

"Kodun ilk yüzde **90**'ını yazmak geliştirme zamanının yüzde **90**'ını alır.

Kodun geri kalan yüzde **10** kısmını yazmak ise geliştirme zamanının diğer yüzde **90**'ını alır."

Analiz ve Planlama Aşaması

- ☞ **Günde kaç satır yazabilirim, Projeyi ne kadar sürede bitirebilirim?**
 - Normalde bir kişinin günde 50-60 satır kod yazabileceği öngörülür, fakat bu sayı karmaşık yapıları olmayan kodlar için geçerlidir. Ortalamada ise kişinin günde ortalama bir fonksiyon yazıp, biraz da test edebileceği öngörülür. Yani ortalama 30 satırdan, yaklaşık ayda 1000 satır.
 - Ancak çoğunlukla bu sayı tatiller, kod yazılmayan analiz, test, entegrasyon gibi süreçleri çıkarırsak ortalama ayda 400 satıra düşer.
 - Elbette 1000 satır kod üretip sonra aylarca çıkan sorunları çözmeye çalışan proje ekipleri de mevcuttur.
- ☞ **Kendinizden emin değilseniz proje içi ayırdığınız süreyi Pi sayısı (3,14) ile çarpın**
 - Proje süresi tahmini için birden fazla yöntem kullanmamış ve çok detaylı analizler yapmamışsanız proje süresi için yaptığınız tahmin Pi sayısı ile çarpın.
 - Genelde tahmin yapılırken sadece tasarım ve kodlama süreleri dikkate alınır ancak; entegrasyon, kodun test edileme, sahaya kurulma, eğitim hazırlama ve verilmesi, dokümanların hazırlanması, elemanların ani ayrılması veya hastalanmaları ve tatil süreleri de proje süresini etkiler.

Analiz ve Planlama Aşaması

Proje Büyüklüğünü Tahmin Etmek

- **Benzerlik Yöntemi** – Projeyi daha önce yapılmış projeler ile kıyaslayarak tahminde bulunma yöntemidir. Yöntemi kullanabilmek için daha önceden yapılmış kıyaslanabilecek projelerin olması gereklidir.
- **Uzman Görüşü** – Proje tahmininde uzmanlardan görüş alınır. Sonuç değişken olabilir ve uzmanın tahmin ve konusundaki ustalığına bağlıdır.
- **Tepeden Aşağı İnceleyerek Tahminde Bulunma Yöntemi** – Sistem en tepeden ve en genelden başlanıp detaylara inerek incelenir. Bu yöntemde sistemin tanımının ve gereksinimlerinin çok iyi belirlenmiş olması gerekir. Yöntemin dezavantajı, verilecek kararların mühendislik grubunun başlangıçtaki düşünce ve tercihlerine çok bağlı olmasıdır.

Analiz ve Planlama Aşaması

🌀 Proje Büyüklüğünü Tahmin Etmek (devam)

- **Aşağıdan Tepeye İnceleyerek Tahminde Bulunma Yöntemi** – Sistem en küçük parçalarından başlanarak incelenir. Detaylı, zaman alıcı bir yöntemdir. Üstelik parçaların büyüklüğü hesaplanırken bu parçaların birleştirme maliyeti genelde unutulur. Bu yöntem kullanılarak projede bir süre yol alındıktan sonra ilk parçaların yapılma süresi gözlenir. Orantı kurularak tüm parçaların ne kadar zamanda gerçekleşeceği hesaplanabilir ve ekibin ilerleme hızı ve dolayısıyla projenin süresi gerçekçi bir biçimde hesaplanabilir.
- **Maliyete Göre Tasarım** – Bu yöntemde önerilen bütçeye göre ne yapılabilecekse o tahmin edilip yapılmaya çalışılır. Mühendislik temelinden uzak bir yöntemdir.
- **Parametrik modeller kullanmak** – Bu yöntemde detaylı mühendislik hesapları yapılır ve tahmin yöntemleri kullanılır. Avantajı şudur; hesaba dayalı bir işlem olduğu için erken sonuçlar alınabilir ve sonuçlarda objektif olunabilir. Ancak doğru uygulanmadığında ya da insanlar yöntemi suistimal ettiğinde çok iyimser tahminler üretebilir.

Analiz ve Planlama Aşaması

☞ Parametrik modeller kullanılarak yapılan tahminlerin örnekleri:

- Kaynak kod satırı sayısı ile proje büyüklüğü tahmini
- İşlev noktaları sayısı ölçümü ile proje büyüklüğü tahmini
- İş ana parçalara bölerek proje büyüklüğü tahmini
- Nesne temelli programlama yapılırken kullanılan yöntemler
 - Nesne öğeleri sayısı temelli tahmin
 - Kullanım senaryoları (Use Case) sayısı temelli tahmin
- COCOMO Tahmin Modeli

☞ Kalite, Hız, Ucuzluk Üçgeni

- Ürün kaliteli ve hızlı yapılırsa ucuz olamaz,
- Ürün kaliteli ve ucuz yapılırsa hızlı yapılamaz,
- Ürün hızlı ve ucuz yapılırsa kaliteli olamaz.

Analiz ve Planlama Aşaması

☞ Planlama ve İyi Bir Planlamanın Önemi

- Proje küçük de olsa mutlaka bir plan yapılmalıdır
- Plan içerisinde yapılacak ana işler, görev dağılımı, görevlerin başlangıç ve bitiş süreleri yazılmaya gayret edilmelidir.

☞ Proje Adımlarını Belirleyin

- Proje adımının tanımı
- Bu adıma ulaşmak için gerekli süre
- Bu adıma ulaşmak için gerekli efor ve elemanlar
- Proje adımının başarı kriteri, yani bu adıma başarıyla ulaşıldığının objektif belirlenmesi,
- Aşamaların öncülleri, ardılları.

☞ Proje adımlarını dokümente ederken birçok yöntem vardır. Pert çizelgeleri (Program Evaluation and Review Technique - İş Değerlendirme ve Gözden Geçirme Tekniği), CPM Çizelgeleri (Critical Path Method - Kritik Patika Yöntemi), Gantt Çizelgeleri (Gantt Chart) bunlardan bazılarıdır. Özellikle Gantt Çizelgeleri yazılım projelerini safhalara ayırırken olmazsa olmaz yöntemlerden biridir. Gantt çizelgeleri basitçe iş parçalarını ve bunların tahmini sürelerini aynı çizelge üzerinde, zaman ekseninde göstermekten ibarettir.

Analiz ve Planlama Aşaması

∞ Proje İçin Gerekli Olan Kaynaklar

- Herhangi bir projeye başlayabilmek için aşağıdaki dört kaynak olmazsa olmaz unsurlardır:
 - İşgücü
 - Donanım
 - Zaman
 - Para
- Bunun yanısıra aşağıdakiler de başarılı olmak için neredeyse vazgeçilmezdir:
 - Proje kapsam tanımı- Project Scope (Proje amacı, büyüklüğü, gereksinimleri, paydaşlarının belirlenmesi ve başarı kriteri)
 - Motivasyon
 - Sağlıklı çalışma ortamı
 - Kullanıcı geri beslemesi

Analiz ve Planlama Aşaması

Risk Analizi

- Yazılım projeleri risk oranı yüksek projelerdir. Bu yüzden başarısızlık oranları bir hayli yüksektir. Başarılı projeler:
 - Zamanında tamamlanmış
 - Bütçesini aşmamış
 - Belirlenen gereksinimleri karşılayan projelerdir.

Analiz ve Planlama Aşaması

∞ Yazılım Projelerinde En Çok Rastlanan Riskler Nelerdir?

- Proje amaç, tanım ve kapsam riskleri
- Gereksinim riskleri
- Takvim riskleri
- Bütçe ve maliyet riskleri
- Planlama riskleri
- Gerçekleme riskleri
- Kalite güvence riskleri
- Kalite kontrol riskleri
- Organizasyondan kaynaklanan riskler
- Yönetimsel riskler
- Müşteri ve kullanıcılarla ilgili riskler
- Paydaşsal riskler
- Altyüklenici riskleri
- Sosyal riskler ve personel riskleri
- Teknolojik riskler
- Ticari riskler
- Politik ve yasal riskler
- Toplumsal riskler

Analiz ve Planlama Aşaması

☞ Riskleri Azaltmak İçin Ne Yapılmalıdır?

☞ **Sorun:** Gerçekçi olmayan takvim ve bütçeler.

☞ **Çözüm:** Takvimleri hazırlarken kodlamayı yapacak personelin fikrine mutlaka başvurunuz. Bütçe hazırlarken geçmiş projelerden faydalanmayı ihmal etmeyiniz. Bunun için bir proje veritabanı tutmanızın ve proje sonu değerlendirme çalışmaları yapmanızın gerektiğini bir kez daha hatırlatalım. Takvim aşılması sorununu yaşamamak için iletişim kanallarının mümkün olduğunca açık olması ana şarttır. Ayrıca projede belirsizlik ne kadar fazla ise o kadar çok “tekrar işlemeli/adımsal” (iteratif) ve “artırımsal” incremental) yaklaşımlara yönelmeli, böylece projeyi denetlenebilir safhalara ayırmalısınız.

☞ **Sorun:** Personel sıkıntısı veya sık değişimi.

☞ **Çözüm:** Personel değişiminin zararlarından korunmak için tavsiye edebileceğimiz iki yöntem vardır. Birincisi tahmin edilebileceği gibi personeli memnun etmektir. İkinci yöntem ise personelin ne yapılsa yapılsın değişebileceğini doğal kabul edip çevik yöntemlerin önerdiği “çiftler halinde programlama” gibi personel kaybı olduğunda bilgi kaybı oluşmasını da engelleyici önlemler almaktır. Kritik bilgilerin birden fazla personel tarafından bilinmesi yararlıdır.

Analiz ve Planlama Aşaması

Risklerin Erken Tespiti Hayat Kurtarır

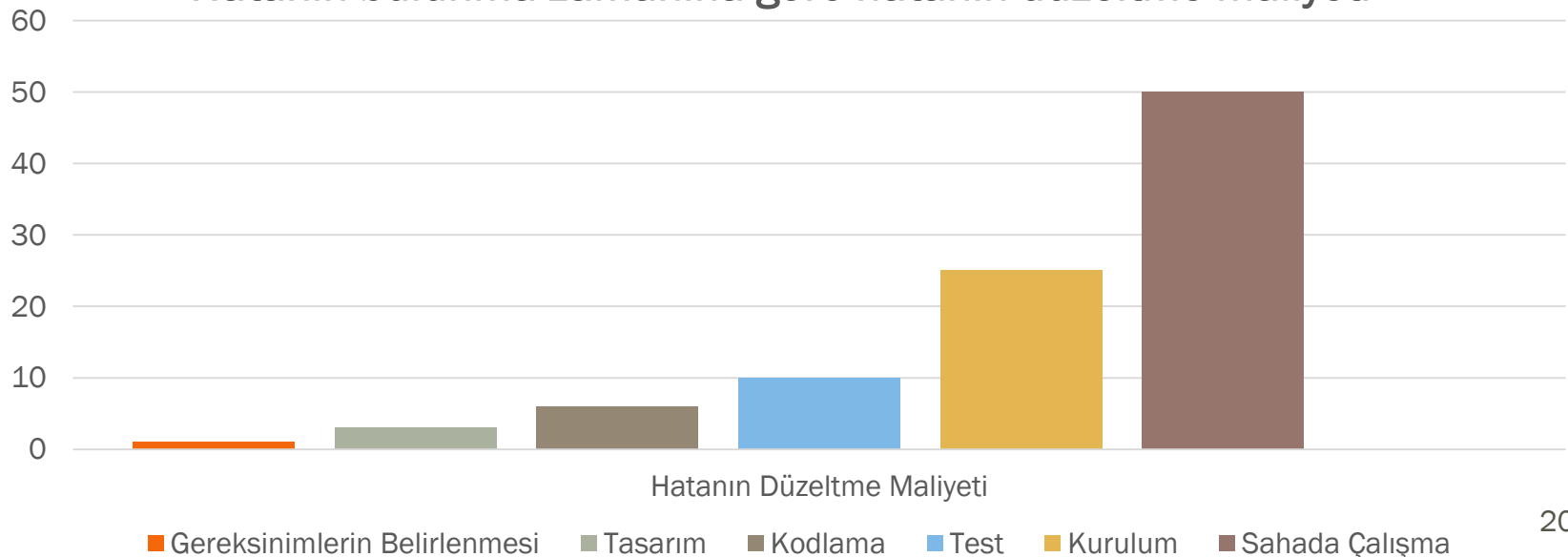
- Riskleri erken tespit etmek için gerekli öğeleri sayarsak:
 - Proje ekibinin iletişiminin iyi olması,
 - Proje içerisinde bilgi akışının iyi olması,
 - Deneyimli teknik proje yöneticisi,
 - Risk yönetimi için görevlendirilmiş bir ekip veya program yöneticisi,
 - Eski projelerin verilerinden faydalanmak,
 - Tahmin yöntemleri kullanmak,
 - Yetki ve sorumlulukları proje başında iyi belirlemek,
 - Proje vizyonunu ve iş tanımını yazılı ve net biçimde yapmak,
 - Proje gereksinimlerini proje boyunca her mümkün olduğu anda daha da netleştirmek,
 - Proje tipi daha belirsiz ise daha da iteratif (tekrar işlemeli/adımsal) yöntemler kullanmak.

Gereksinimlerin Belirlenmesi Aşaması

☞ Gereksinimlerin Belirlenmesi Aşaması

- Gereksinimler müşteri ve yazılım ekibi arasında yazılımın ne yapacağı ve hangi özelliklere sahip olacağı konusunda bir anlaşmadır. Gereksinimlerin doğru olarak belirlenmesi çok önemlidir. Gereksinimler sırasında yapılan ve tespit edilemeyen bir hatanın maliyeti sonraki aşamalarda giderek artar.

Hatanın bulunma zamanına göre hatanın düzeltme maliyeti



Gereksinimlerin Belirlenmesi Aşaması

- ✎ Yazılım gereksinim dokümanının faydaları şunlardır:
 - Müşteri ile yazılım ekibi arasındaki yanlış anlamaları yok eder, olası uygunsuzlukların en başta tespit edilmesini sağlar.
 - İşgücü, maliyet ve süre analizlerine ek bir temel teşkil eder ve doğru yapılmalarını sağlar,
 - Testler ve kabul kriterleri için temel teşkil eder,
 - Diğer dokümantasyonlara temel teşkil eder,

- ✎ Gereksinimler ile ilişkili standartlar şunlardır:
 - IEEE STD 1233-1998 Sistem Gereksinimleri için tavsiyeler
 - IEEE STD 830-1998 Yazılım Gereksinimleri için tavsiyeler
 - MIL-STD-498 Askeri Yazılım Geliştirme Standardı

Gereksinimlerin Belirlenmesi Aşaması

❧ İyi Bir Gereksinim Dokümanı Hangi Özelliklere Sahip Olmalıdır?

- **Doğru** - Elbette gereksinim dokümanınızın doğru olmasını istersiniz. Ancak önemli olan güncel tutarak uzun süre doğru kalmasını sağlamaktır.
- **Tartışmasız** - : Bir gereksinim herkes tarafından aynı şekilde yorumlanmalıdır. Bir gereksinimden herkes farklı bir şey anlamamalıdır.
- **Tam** - Yazılımın gerçekleşmesi için gerekli tüm konulara ait gereksinimlere değinmelidir.
- **Tutarlı** - Gereksinim dokümanı içindeki her madde diğer maddeler ile uyumlu olmalıdır. Gerek mantık olarak gerekse yazım biçimi olarak uyum sağlanmalıdır.
- **Öncelik sıralı** - Gereksinimler gerçekleşme önceliklerine göre gruplanır ve sıralanırsa proje takvimi açısından çok değerli bir iş yapılmış olur. Önemli gereksinimler önce, daha az önemliler daha sonra gerçekleşir. Gereksinim dokümanında öncelik bilgisi de verilmelidir.

Gereksinimlerin Belirlenmesi Aşaması

☞ İyi Bir Gereksinim Dokümanı Hangi Özelliklere Sahip Olmalıdır? (devam)

- **Doğrulanabilir** - Gereksinim olarak doğrulanması güç, muğlak ifadeler kullanmayınız. Örneğin “Sistem hızlı cevap verecektir.” ifadesi yerine “Sistem alınan mesajlara en fazla 100 ms içerisinde cevap mesajı gönderecektir.” türünden ölçülebilir ifadeler yazın.
- **Değişikliğe uygun** - Gereksinim dokümanı içerisindeki gereksinimler ve açıklamalar basit, ufak ve mümkün olduğunca birbirlerinden bağımsız olmalıdır. Ayrıca bir gereksinim birçok yerde tekrarlanmamalıdır. Böylece gereksinim dokümanı değiştirilmek istendiğinde kolayca değiştirilebilir.
- **İzlenebilir** - Gereksinimlerin gerek sistem gereksinimlerine gerekse testlere olan ilgileri açıkça belirtilmelidir. Bir yazılım gereksiniminin hangi sistem gereksiniminden doğduğu bilinirse gerçekleşme daha tutarlı ve hatasız yapılabilir. Öte yandan hangi testin bu gereksinimi test ettiği bir yerde yazılı olursa testler esnasında gözden kaçan yani test edilmemiş gereksinim kalmaz.
- **Gerçeklenebilir** - Gereksinimlerin gerçekten gerçekleştirilebilir olduğunun önceden düşünülmesi faydalıdır. Gereksinimler yazılırken bile bir ön araştırma yapıp bu gereksinimin eldeki imkânlarla başarıyla gerçekleştirilemeyeceği düşünülmelidir.

Gereksinimlerin Belirlenmesi Aşaması

🌀 Gereksinimleri Hazırlarken ve Yazarken Nelere Dikkat Etmek Gerekir?

- Gereksinimleri hazırlama safhasında aşağıdaki konulara dikkat etmelisiniz:
 - Öncelikle birçok projede kullanabileceğiniz standart bir gereksinim dokümanı şablonunuz olsun.
 - Müşteri ve konunun uzmanları ile sıkı bir iletişimde bulunun.
 - Proje süresinin %20 ile %25'i gereksinimler için harcanmalıdır.
 - Gereksinim dokümanını güncel tutun.
 - Gereksinim dokümanını gözden geçirme işlemine sokun, başka insanlara okutun, anlaşılıp anlaşılmadığını kontrol edin

Gereksinimlerin Belirlenmesi Aşaması



Gereksinimleri yazarken aşağıdaki konulara dikkat etmelisiniz:

- Tümcelerinizi kısa tutun,
- Edilgen tümceler kullanmamaya çalışın, çünkü edilgen yapıda özne olmadığı için sorumluluğun kimde olduğu belli değildir,
- Gramer kurallarına dikkat edin,
- Gereksinim dokümanı içerisinde geçen terimler için terimler sözlüğü oluşturun,
- Gereksinim dokümanı içerisinde geçen kısaltmalar için kısaltmalar sözlüğü oluşturun,
- Gereksinim dokümanı içerisinde anlaşılması zor veya teknik olan konuları tanım maddeleri olarak verin. Örneğin “Arayüz” kelimesi için “İki veya daha fazla cihaz arasında veri iletişiminin sağlandığı donanım birimi” gibi bir açıklama tüm paydaşları çok memnun edecektir,
- Tek bir gereksinim maddesini test etmek için üç-dört testten fazla test yazmak zorunda kalıyorsanız muhtemelen o gereksinimi aslında birkaç parçaya bölmeniz gerekiyordur

Gereksinimlerin Belirlenmesi Aşaması

☞ Gereksinimleri yazarken aşağıdaki konulara dikkat etmelisiniz (devam):

- Ve/Veya kelimelerinin fazla geçtiği gereksinim maddeleri genelde özensizce yazılmış maddelerdir,
- Negatif anlamli tümcelerden kaçının,
- Gereksinimlerin seviyeleri birbirine yakın olmalıdır. Açıklayacak olursak bir gereksinim tüm sistemi tarif ederken diğer bir gereksinim bir detayı tarif ediyorsa bir sorun var demektir,
- Tekrar eden gereksinimlerden kaçının. Sadece açıklama kolaylığı sebebi ile mecbur kalındığında tekrar eden gereksinimler kullanılmalıdır. Aksi takdirde tekrar eden gereksinimler dokümanın bakım yapılabilirliğini fena halde azaltırlar,
- Gereksinimi yazan kişiler ile kodlamayı yapan kişiler mümkünse ayrı kişiler olsunlar,
- Gereksinimi yazan kişilerin dil yeteneği ve iletişim becerisi çok gelişmiş olmalıdır,
- Tam anlaşılmayan veya belirsiz gereksinimlerin ileride size sorun çıkaracağından emin olabilirsiniz,

Gereksinimlerin Belirlenmesi Aşaması

☞ Gereksinimleri yazarken aşağıdaki konulara dikkat etmelisiniz (devam):

- Yazılım gereksinimlerini sistem gereksinimlerine (veya ana kaynak üst gereksinimlere) bağlayan izlenebilirlik tablosu (matrisi) oluşturun,
- Gerçekleme yöntemi ve tasarım detaylarını gereksinimler içine koymaktan kaçının,
- Bir gereksinim maddesinde sadece bir gereksinimi belirtin,
- Gereksinimi yazarken nasıl test edileceğini de düşünün,
- Gereksinimin gerçek dünyada sağlanıp sağlanamayacağını kontrol edin,
- Sadece normal çalışmayı değil istisnai durumlarda alınacak tutumu da gereksinimlerde belirtin,
 - Sisteme uygun olmayan girdi girildiğinde sistemin davranışını belirtin,
 - Sisteme (beklediği) girdi girilmediğinde sistemin davranışını belirtin,
 - Sisteme beklenmedik bir bilgi beklenmedik bir zamanda girildiğinde sistemin davranışını belirtin,
- Sistemin başlatılma ve kapatılma süreçlerindeki davranışını belirtin,
- Gereksinimlerin sistemin hangi mod ve durumlarına ait olduğunu belirtin.

Gereksinimlerin Belirlenmesi Aşaması

☞ Peki bir gereksinim dokümanı neyi içermelidir?

- **İşlev:** Yazılımın ne yapması isteniyor? Amacı nedir?
- **Kullanım senaryoları (Use Cases):** Modern gereksinim dokümanları insan-makina arasındaki kullanım senaryolarını da içerir.
- **İş akışı:** Hem modern hem de klasik gereksinim dokümanlarında akış senaryoları, cihazların iletişim protokolleri, mesajlaşma akışına dair bilgiler yer alır.
- **Dış arayüzler:**
 - Yazılımın insanlarla olan arayüzü (ekranlar, sesler, vb)
 - Yazılımın diğer cihazlarla, sistem donanımı ile ve diğer yazılımlarla olan arayüzü.
- **Performans:** Hız, düzgün çalışma oranı, cevap verme süresi, vb.
- **Yazılımın özellikleri:** Taşınabilirlik, doğruluk, bakımı yapılabilirlik, güvenlik, vb kriterler.
- **Gerçekleme üzerindeki kriter ve kısıtlar:** Uygulanan standartlar, uygulama yazılım dilinin adı, veritabanı politikası, kaynak sınırlamaları, işletim ortamı, vb.
- **İşe özel kurallar:** Bu kısımda işin karakteristikleri, kendine has özellikleri anlatılır.
- **Diyagramlar:** Gereksinim dokümanında sadece gereksinim maddeleri bulunmaz, ayrıca teknik detayları veya büyük resmi daha iyi anlamaya yarayan şemalar, şekiller, akış diyagramları da bulunmalıdır.

Gereksinimlerin Belirlenmesi Aşaması

🌀 **Örnek: Açık Bir Gereksinim Maddesi Hazırlamak**

İstek 1: Frene basınca araba dursun.

İlk deneme "geçiş, zaman belirsizliğine" sahip bir gereksinim maddesi:

Gerek 1: Frene basınca araba duracaktır.

İtiraz 1: Ne zaman duracak? Basar basmaz mı? Nasıl olacak o iş?

Aşağıda da iki gereksinim belirtilmiş, üstelik eksikler de var:

Gerek 1: Fren pedalına basınca fren pedalı aşağıya inecek ve araba duracaktır.

İtiraz 1: Fren pedalı aşağıya inmezse ne olacak? Aşağıya inmesi ile durmanın ne ilgisi var?

Gereksinimlerin Belirlenmesi Aşaması

Aşağıda da koşul kalıbı belirsiz bir gereksinim var.

Gerek 1: Araba duracaktır.

Veya Gerek 1: Araba gerektiğinde duracaktır.

İtiraz 1: Gerektiğinde ne demek? Hangi koşulda duracak?

Bu sefer de başvuru belirsizliği sorununa sahip kötü hazırlanmış gereksinim örneği:

Gerek 1: Araba bazı durumlarda duracaktır. Bu durumlar için Bölüm 1'e bakın.

İtiraz 1: Bölüm 1'de baktım çok bilgi var, nerede bu "bazı durumlar"?

Daha da yanlış gereksinim örneği, yine başvuru belirsizliği var.

Gerek 1: Araba bazı durumlarda duracaktır. Bu durumlar için frenle ilgili bölüme bakın.

İtiraz 1: Frenle ilgili bölüme mi bakayım? Ama 100 sayfalık dokümanın yarısı frenle ilgili zaten. Konu başlığında frenle ilgili mi yazıyor?

Gereksinimlerin Belirlenmesi Aşaması

Şimdi de tasarımın gereksinimlerde yapılması hatasını görelim.

Gerek 1: Fren pedalına basınca hidrolikler harekete geçecektir.

Gerek 1: Hidrolikler harekete geçince tekerleklerle basınç iletilecektir.

Gerek 1: Hidroliklerin uyguladığı basınç tekerlekleri kilitleyecek ve böylece araba duracaktır.

İtiraz 1: Peki ya fren mekanik olarak gerçekleşirse ne olacak?
Neden hidrolik kısıtlaması koyalım? Bırakalım işin nasıl yapılacağını tasarımcı düşünsün.

Şimdi de iyi hazırlanmış bir gereksinim görelim.

Gerek 1: Fren pedalına basınca tekerlekler 7 saniye içerisinde duracaktır.

Ön ve Detaylı Tasarım Aşaması

- ✎ Bu aşamada yazılımın içerdiği tüm bileşenler ayrıntıları ile tanımlanır ve tasarlanır.
- ✎ Tasarım aşaması zaman bakımından kritik bir aşamadır. Gereğinden az zaman harcamak ileride büyük sorunlar doğurur.
- ✎ Öte yandan bu aşamada gereğinden fazla zaman harcamak projenin başarılabilmesini riske sokar.

Ön ve Detaylı Tasarım Aşaması

☞ Kavramsal Sistem Tasarım Dokümanı Hazırlayın

- Kavramsal sistem tasarım dokümanı gereksinimler, kullanım senaryoları ve iş tanımından yola çıkılarak hazırlanan ve sistem tasarımını büyük resimden başlayıp daha detaylara inerek anlatan dokümandır.
- Öncelikle bu dokümanda sistemin somut ve soyut elemanlarını, onların birbirleri ile bağlarını, sistemin dış dünya ile olan bağlarını anlatan sistem mimari planı bulunmalıdır.
- Eğer varsa sistemin ekran pencerelerinin resimleri, rapor formatları bu dokümanda belirtilir.
- Bu dokümanın birçok işlevi vardır, bunlar:
 - Sistemin ana tasarımının çatısını oluşturmak, yanlış tasarımın önüne geçmek,
 - Diğer yazılım ve donanım detay dokümanlarının hazırlanmasına temel teşkil etmek,
 - Yeni başlayan proje elemanlarının yetişmesine ve projeye adapte olmasına yardımcı olmak,
 - Eğitimlerin ve eğitim dokümanlarının hazırlanmasını kolaylaştırmak,
 - Know-How'ı arttırarak sistemin bakımını kolaylaştırmak,
 - Organizasyonun teknik ve teknik olmayan grupları arasında köprü görevi görmek

Ön ve Detaylı Tasarım Aşaması

🌀 Projenin Sorumlulukları ve İş Dağılımı Çizelgesi Hazırlayın

- Büyük projelerde en büyük sorunlardan biri kimin hangi işi yaptığının belli olmamasıdır.
- Özellikle bürü projelerde sorumluluklar ve iş dağılımı çizelgesinin (matrisinin) olmaması önemli bir sorun olarak karşımıza çıkar.
- Böyle bir çizelge olmaması durumunda:
 - Proje ekibi ile iş yapmak durumunda olanlar kiminle irtibat kuracaklarını bilemezler,
 - İşe yeni başlayanlar bilgiye kimden ulaşacaklarını bilemezler,
 - İş vermek isteyen amirler iş yükünü ayarlayamazlar, doğru kişiye doğru işi veremezler,
 - Müşterinin sorunu bir türlü doğru teknik kişiye ulaşamadığından çözüm süresi bir hayli uzar.

Ön ve Detaylı Tasarım Aşaması

- ☞ Bu çizelge birkaç şekilde hazırlanabilir. Örneğin en basit halinde:

Proje Lideri	Ali
Kontrol Yazılımı	Veli
Ekran Tasarımı	Mehmet
Haberleşme Altyapısı	Ayşe

- ☞ Diğer bir şekilde:

	Ali	Veli	Mehmet	Ayşe
Proje Lideri	Asıl	Yedek	-	-
Kontrol Yazılımı	-	Asıl	Destek	-
Ekran Tasarımı	-	-	Asıl	Yedek
Haberleşme Altyapısı	-	Yedek	-	Asıl

Ön ve Detaylı Tasarım Aşaması

☞ Daha organizasyonel şekilde:

Proje Sahibi	Selim
Proje Yöneticisi	Salih
Proje Teknik Lideri	Ali
Proje Yazılım Ekibi	Veli, Mehmet, Ayşe
Müşteri İlişkileri	Can
Konfigürasyon Sorumlusu	Derya
Satınalma Sorumlusu	Fatma
Müşterinin Temsilcisi	Hasan

☞ Ayrıca ağaç dallanması şeklinde hiyerarşik proje sorumluluk çizelgeleri de hazırlanabilir.

Ön ve Detaylı Tasarım Aşaması

🌀 Terimler ve Kısaltmalar Sözlüğü Hazırlayın, kullanacağınız Birimleri/Limitleri Önceden Belirleyin

- Özellikle büyük ve çok insanın çalıştığı projelerde proje başladığında yapılması gerçekten çok faydalı olan bazı alışkanlıklar vardır.
- Bunların bir kısmını verecek olursak:
 - Tüm projede geçen kavramları, isimleri açıklayan bir terimler sözlüğü; örneğin sistemlerin adları, sistemin yaptığı önemli işlerin açıklamaları, vb.
Not: Bunun güncel tutulması çok önemlidir.
 - Projedeki kısaltmaların açılımları.
 - Projede kullanılan birimler kılavuzu; örneğin uzaklık birimi olarak Kilometre mi yoksa metre mi kullanılacağı, ağırlık olarak Kilogram mı yoksa Pound mu kullanılacağı, hız ölçüsü olarak Metre/Saniye mi yoksa Km/Saat mi kullanılacağı gibi.
 - Birim zaman belirlenmesi ve kılavuzu; yani sistemin zaman bilgisinin ve artışının nasıl çalışacağını belli olması; örneğin her birim 10 milisaniyedir, başlangıç saati olarak Unix saati alınmıştır, göreceli saat olarak maksimum 200000 birimde tur bindirilir ve bu 4 bayt içinde tutulur gibi.
Not: Yıl 2000 problemi benzeri problemler bir daha olmayacak ve sizin başınıza gelmez zannetmeyiniz

Ön ve Detaylı Tasarım Aşaması

🌀 Kodlama Standartları ve Tavsiyeleri Oluşturup Onları Kullanın

- Koddaki hataları mümkün olan en erken zamanda tespit etmenin hata düzeltme maliyetini 5 ila 100 kat düşürdüğü artık günümüzde bilinen bir gerçektir.
- Hataları zamanında tespit etmenin en kolay ve verimli yollarından biri de kod gözden geçirme (code review) toplantılarıdır.
- Kod gözden geçirme toplantıları yapabilmek için standart bir kod üretilmesi gereklidir

Ön ve Detaylı Tasarım Aşaması

☞ Kodlama standartlarının faydalarını inceleyecek olursak, standartlara uyulduğunda:

- **Bakım yapılabilirlik artar:** Kodlama standartları okunabilirliği büyük ölçüde artırır ve karmaşıklığı azaltır. Bu sebeple kod yazıldıktan çok sonra bile kodu anlamak ve hatalarını çözmek kolay olur.
- **Yazılım elemanı bulmak kolaylaşır:** Evet yanlış duymadınız kod standartlarına uymak kodun karmaşıklığını azalttığı ve okunabilirliğini çoğalttığı için daha az kalifiye bir yazılımcı da bu kodda eklemeler ve düzeltmeler yapabilir. Yetişmiş bir yazılımcı bulmak çok zordur, kod standartlarına uyulduğunda daha az tecrübeli bir yazılımcı da rahatça bu kod üzerinde çalışabilir.
- **Kodun tekrar kullanımı kolaylaşır:** Kodun anlaşılabilirliği arttığı için başka projelerde bu kodu kullanmak da kolaylaşır. Normal koşullarda bu tekrar kullanılan kodun eskiden birçok teste tabi tutulmuş olduğunu kabul edersek ve şimdi eklendiği yerde de testlere tabi tutulacağını düşünersek koddaki hata oranı bayağı düşük olur.
- **Kalite kuruluşlarından onay almak kolaylaşır:** Standartlara uymanın dolaylı bir yararı da kalite kuruluşlarının denetlemelerinde daha yüksek puan alabilmek ve kalite belgelerine güvenilir biçimde ulaşabilmektir.
- **Dokümantasyon kolaylaşır:** Standartlar yorum satırlarına da bir düzen getirdiği için daha sonra bu yorumlardaki bilgilerden faydalanarak tutarlı dokümanlar hazırlanabilir. Dokümantasyon ile kod arasındaki uyumsuzluklar minimuma iner.

Ön ve Detaylı Tasarım Aşaması

- 🌀 Kodlama standartlarının faydalarını inceleyecek olursak, standartlara uyulduğunda (devam):
- **Otomatik gözden geçirme gereçlerinin işi kolaylaştır:** Standartlara uygun olarak yazılmış bir kodu otomatik programlarla tarayıp değişik işlemler için kullanmak kolaylaştır. Bu işlemler için örnek verirsek; satır sayısının hesaplanması, hataların taranması, kodun karmaşıklığının hesaplanması, değişken ilklendirilmemesi gibi hataların bulunması, vb.
 - **Kodun karmaşıklığı azalır:** Standart kodlarda karmaşık yapıların kullanımına izin verilmeyeceği için kodun karmaşıklığı azalır.
 - **Kodun güvenilirliği artar:** Örnek vererek açıklayacak olursak gömülü sistemlerde dinamik bellek ayrılmasına izin verilmeyeceği için gömülü sistemlerin sahadaki korkulu rüyası olan null işaretçi hatalarına da rastlanmaz, kritik bir sistemin sahadayken çökme ihtimali bir hayli azalır.
 - **Eğitim giderleri düşer:** Projeye her yeni başlayan yazılımcı resmi veya gayri resmi eğitim görür. Yani ya adı konmuş bir eğitim alır ya da kendisi projeyi hatta kullanılan yazılım dilini öğrenmek için çalışır. Oysa standartlara uyularak yazılan bir kod adeta bir eğitim kitabı gibidir; gerek yazılım dili gerekse projenin detayları konusunda yeni başlayan elemana eğitim kaynağı olur.

Ön ve Detaylı Tasarım Aşaması

- ☞ Kodlama standartlarının faydalarını inceleyecek olursak, standartlara uyulduğunda (devam):
 - **Bilgi birikimi sağlar:** Standartlara uyulan bir kod gerek hazır kodlanmış algoritmalarıyla, tasarım desenleri ile gerekse sık yapılan hataların çözümleri ile yeni yazılımcıya önderlik eder.
 - **İzlenebilirlik artar:** Standartlara uyulan kodlarda gerekirse kod parçası- karşıladığı gereksinim gibi özellikler not edilebileceği için gereksinimlerin karşılanıp karşılanmadığını izlemek kolaylaşır.
 - **Farklı kaynaklardan gelen kodları bir arada kullanmak kolaylaşır:** Kodlama standartları uygulandığında isim uzayları (name space) uygulanacağından aynı isimli değişkenlerin çakışması engellenmiş olur. Örneğin A firmasının ve B firmasının kaynak kodlarında “kullanıcıAdı” diye bir değişken olsa bile farklı isim uzaylarında olacakları için derlemede sorun olmayacak ve kodlar birlikte derlenebilecektir.
 - **Koddaki hata miktarı azalır:** Genelde kodlama standartları her yazılım dili için özel olarak hazırlanırlar. Bu sayede, kodun yazımı sırasında oluşan hataların en aza indirilmesi amaçlanır.
- ☞ Kodlama standartlarınızı sadece bir proje için değil tüm projelerde kullanmak üzere oluşturmalsınız. Projenin başından sonuna kadar standartlara uyumu denetlemelisiniz, kodunuzu projenin en sonunda standartlara uydurmaya çalışmamalısınız.

Ön ve Detaylı Tasarım Aşaması

∞ Ne Gerek Var Dokümana Ben Varım ya

- Yazılım geliştirme dokümanları yazılımın tekrar edebilen bir başarıyı yakalayabilmesi için gerekli öğelerinden biridir. Bu dokümanlar sayesinde;
 - Yazılımın takibi kolaylaşır,
 - İş bölümü sağlanır,
 - Yazılımın bakımı kolaylaşır,
 - Yeni elemanların eğitimi kolaylaşır,
 - Yazılımın yönetiminin yapılması, örneğin bütçesinin ve takviminin hazırlanması kolaylaşır,
 - İstatistiki veriler hazırlamak kolaylaşır,
 - Kullanıcıların yazılımı öğrenmesi ve kullanması kolaylaşır.

"Programlama alanında, dokümante edilmemiş bir programdan daha değersiz bir şey yoktur."

Ön ve Detaylı Tasarım Aşaması

- ∞ Yazılım projelerinde standart yayınlayan kuruluşlar vardır ve dokümantasyon biçimi ile ilgili önerilerde bulunurlar.
- ∞ Hepsinin doküman önerilerindeki ortak noktalarını bulmaya çalışırsak bir yazılım projesinde hazırlanan dokümanlar şunlardır;
 - Süreç dokümanları:
 - Yazılım standartları
 - Raporlar, hata bildirim formları, iş yapma tarif formları,
 - Kalıcı teknik kaynaklar, kitaplar, makaleler,
 - Anlık teknik kaynaklar, e-mailler, vb.
 - Proje ve Ürün Dokümanları
 - Gereksinim dokümanı
 - İşlevsel tasarım dokümanı
 - Detaylı tasarım dokümanı
 - Test dokümanı
 - Test raporu

Ön ve Detaylı Tasarım Aşaması

- ✎ Yazılım geliştirirken hazırlanacak dokümanların uyması gereken biçimleri ve niteliklerine ilişkin standartlar geliştirilmiştir.
 - IEEE (Institute for Electrical and Electronic Engineering) enstitüsünün "**IEEE Standard for Software User Documentation**" isimli standardında dokümanların hazırlanmasına dair genel kurallar anlatılır.
 - Yine ISO (The International Standards Organization) organizasyonu ve IEC (The International Electrotechnical Commission) komisyonu **ISO/IEC 18019-2004 (Guidelines for the design and preparation of user documentation for application software)** yayınında içerik ve niteliğe ait kurallar ve **ISO/IEC TR 9294 (Guidelines for the management of software documentation)** yayınında doküman yönetimine ait kurallar bulunabilir.
- ✎ Yeni çıkan "**ISO/IEC 26514:2008 "Systems and software engineering - Requirements for designers and developers of user documentation"**" da doküman hazırlanırken dikkat edilmesi gereken kuralları anlatır.

" Siz bir düşüncenizi bir yere yazana kadar o düşünce bir fikir değildir."